



OUTLIERS

Operations

Layer 4 of 4 · Running on its own, in order,
and telling you when it stops.

What is it doing while you are not looking?

A system left running fails quietly by default. It carries on, reports success, produces nothing, and there is nobody sitting beside you to notice.

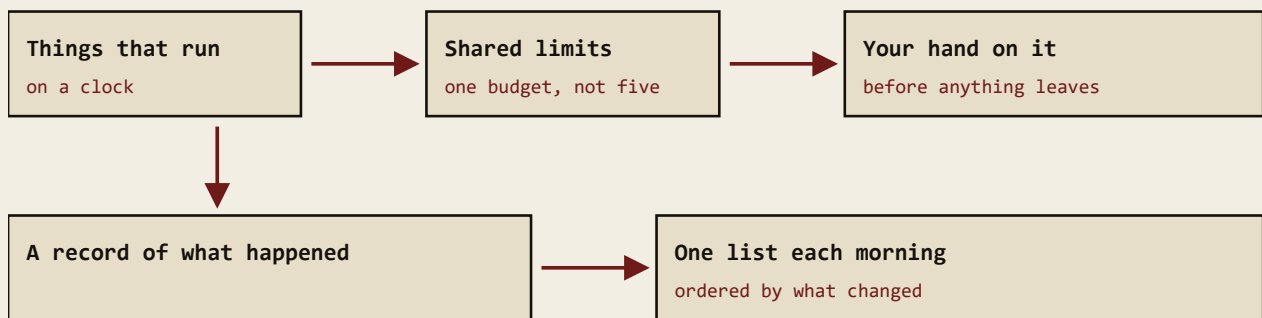
WHAT THE LAST LAYER LEFT YOU WITH

You have assistants that can read your material, form a view, and write. They run when you ask, in whatever order you happen to ask, with nothing shared between them and nothing standing between them and another person.

WHAT THIS LAYER BUILDS

Three things. An order - what runs when, and what is not allowed to run at the same time as something else. A gate - nothing reaches another human being without your hand on it, built into the machinery rather than requested in the instructions. And a record of what actually happened, which is what makes 'is this working' a question with an answer.

Then one short list each morning, ordered by what changed, with a line beside each saying why it is there.



Nothing appears on your screen while you work. A job that steals focus gets switched off within a week.

ORDER, A GATE, A RECORD, AND ONE LIST

How it works

The two failures worth engineering against are the loud one and the silent one.

Order. Jobs run to a timetable rather than when you remember. They run out of sight - no window opening, nothing taking your keyboard mid-sentence. This sounds cosmetic and is not: a job that interrupts you gets turned off, and a job that is off is a job that is not running.

Shared limits. Where several things could act on the same outside world, they draw on one budget between them rather than one each. Five assistants each politely limited to a small number still add up to an unreasonable number.

The gate. Anything intended for another person is written, checked by something other than the thing that wrote it, and then stops. You send it. Nothing here has ever needed to be the thing that presses send, and the moment it is, the whole system's mistakes become other people's problem.

The record. A running list of what happened and when, added to and never overwritten. Notes describe how things are; the record describes what occurred. Without it, 'what changed this week' and 'did that job actually run' are guesses.

The silent failure. A job that ran and produced nothing looks exactly like a job with nothing to do. Mine ran sixty-four times producing nothing and reporting success, and I did not know until something compared what should have happened against what did. That comparison is the last piece, and it is why this layer ends with a check rather than a feature.

Build it

One command. It asks you a few questions so what you get is yours, then does the work.

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-04-operations
cd outliers-sb-04-operations
python install.py
```

This layer needs Layers 1 to 3. It arranges things that already work; there has to be something to arrange.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
When do you want the morning list?	Sets the time the list is ready. It is prepared quietly beforehand, not while you are reading it.
Your hand on anything that reaches a person?	Yes is the default and the recommendation. No is possible and is a decision you should make deliberately, not by accepting a default.
Where should it tell you when something breaks?	A file, a note, or a message. Somewhere you already look, because a warning somewhere you do not look is not a warning.

WHAT YOU END UP WITH

A timetable, a shared limit, a gate before anything leaves, a record that is added to and never rewritten, one list each morning, and a check that compares what should be true against what is - and says so, loudly, when they differ.

Using it

Read one list in the morning. Look at the warnings when they come.

The list is short on purpose and each line says why it is there. It is ordered by what has changed rather than by what you added first, because a reply that arrived this morning matters more than something that has been sitting in a queue for a fortnight.

The warnings are rarer and worth more. Each one names a thing that should be true, what was actually found, and where to look. A warning you cannot act on is a bug in the warning, not something to get used to.

IF YOU ALREADY HAD SOMETHING LIKE THIS

If you already have things running on a schedule, the test is whether you could name which of them are meant to be running right now. Most people cannot, because a job switched off deliberately and a job that broke look identical from outside. This layer writes down which are meant to be on, and then checks.

WHAT THIS LAYER LEAVES UNSOLVED

Nothing further to install. What changes from here is that the system starts telling you things, and what it tells you goes back down the ladder: a rule that keeps misfiring gets rewritten in Layer 2, a source that never produces anything useful gets dropped in Layer 3.

The other thing that changes is what you can now build on top. A foundation that remembers, keeps itself honest, fills itself and runs without watching is the ground another system stands on - and the first one worth standing there is the one that tracks your interactions with other people.

NEXT: LAYER 1 • MEMORY

Back to the bottom with what you have learned. This is the last guide in the set, and not the end of the build: it is the point where the system starts improving from what it reports, rather than from the next instalment.

WORDS USED IN THIS GUIDE

Scheduled task	A job your computer runs at a set time without being asked. Run out of sight, or it interrupts you and gets switched off.
Ledger	A record that is only ever added to, never rewritten, so what it said last month still says the same thing today.
Watchdog	A check that compares what should be true against what is, and complains when they differ.